

Assessment 3	Data-driven application and reporting
Due date	11.59 pm, Sunday 4th October, 2026
Weighting	25%
Word count/length	Video only (3–8 minutes, verbal walkthrough only; no written report required)
SILOs	• Design and develop web applications using JavaScript (SILO 1). • Use modern software engineering tools to build and deploy robust code for scalable web sites (SILO 4). • Develop and deploy secure, reliable full-stack web applications (SILO 2). • Apply appropriate software engineering practices to support cloud-based web systems (SILO 3). • Demonstrate professional judgement in data modelling, testing and analytical presentation (SILO 5).
Graduate capabilities	• INQUIRY AND ANALYSIS – Critical Thinking and Problem Solving • INQUIRY AND ANALYSIS – Research and Evidence-Based Inquiry • COMMUNICATION – Written and Digital Communication • TECHNICAL LITERACY – Digital Tools and Application Development
Generative AI use	Full AI: AI use is accepted for specific tasks such as drafting text, and refining and evaluating work. You are expected to critically evaluate and modify any AI-generated content included. The AI Assessment Scale resource provides guidance on the appropriate use of GenAI tools in assessments and explains what is expected in each category.
Feedback	Verbal feedback will be provided during Assessment 4 Live Presentation. Assessment 3 and 4 will be marked like an exam. Results hidden from students until the results release date.
Individual	This is an individual assessment.
Short extension	Eligible: This assessment is eligible for a short extension. To apply, visit the relevant assessment extension group in Moodle and add yourself to the extension group before the deadline.

Purpose

This assessment is the third stage of the larger project that continues throughout the subject. Assessment 1 focused on frontend design and usability for the Wordle and Word Search builder, and Assessment 2 focused on backend implementation, database integration, CRUD operations, and Docker execution. Assessment 3 now extends the same project into a data-driven web application and reporting stage. The aim is to demonstrate that the Wordle and Word Search builder can store, process, monitor, and present data in a meaningful operational format.

This assessment places a strong emphasis on observability, testing, and reporting. The system should not only work, but also produce evidence that it is functioning correctly, can be monitored, and can be evaluated using practical tests and accessibility checks.

MUST BE CREATED FROM: `npx create-next-app .`

Task details

Develop a data-driven web application that builds on the frontend and backend work completed in Assessments 1 and 2. The application should support stored phoneme-based word lists, dashboard summaries, simulated input records, alerts, reporting views, and operational monitoring. The project should remain focused on the same Wordle and Word Search builder use case, but now the emphasis is on how the system manages, analyses, monitors, and presents information over time.

The application should include meaningful statistics and operational views so that the status of the system can be monitored during use. This must include a healthcheck endpoint where `/health` returns `200 OK`. It should also include database-backed metrics and dashboard indicators for how many Wordle and Word Search activities have been created, average time on page, most-used activity type, successful generation count, failed generation count, and other meaningful usage statistics. These metrics should help show whether the system is active, healthy, and being used effectively.

Instructions

You are required to:

1. **Extend the existing application** so that it includes a data-driven dashboard interface.
 - The dashboard should present useful summaries of the stored word lists and activity configurations.
 - It should include operational information such as health status, how many Wordle and Word Search activities have been created, average time on page, most-used activity type, successful generation count, failed generation count, or other meaningful usage statistics.
 - It should be clear, organised, and appropriate for the builder use case.
2. **Work with simulated input records and database persistence.**
 - The application should store and retrieve records from the database.
 - The data should reflect the structure of the Wordle and Word Search builder, such as word lists, phoneme symbols, activity types, difficulty levels, hints, output settings, and related metadata.
 - The database should also store the statistics used for reporting and observability, such as creation counts, successful generation counts, failed generation counts, average time on page, and activity type usage summaries.
 - The information should be presented in a way that supports reporting and monitoring.
3. **Instrument the application.**
 - Add visible dashboard elements or status indicators that show the behaviour of the application.
 - Include observability features such as a health check, activity creation summaries, usage counters, total generated outputs, average time on page, most-used activity type, or other server-side monitoring outputs.
 - Use these features to show that the app can be monitored and understood while it is running.
 - Show error or warning indicators for failed generation, empty word lists, invalid data, or other unusual states where appropriate.
4. **Test the application using both Playwright and JMeter.**
 - **Playwright:** create end-to-end tests for both the builder and generated output use cases.
 - One Playwright test must demonstrate a **builder use case** such as CRUD operations for a word list or activity configuration.
 - One Playwright test must demonstrate a **user use case** such as generating or viewing a Wordle or Word Search activity.
 - **JMeter:** perform load testing against the builder and generated activity workflow.
 - Your load testing should include multiple traffic levels, such as x1, x10, x100, x1000 and x10000 users, or equivalent staged load levels.
 - Use the results to explain how the system behaves under different loads.
5. **Evaluate accessibility using Lighthouse.**
 - Run Lighthouse accessibility checks and show the results in the video.
 - Explain any changes you made after reviewing the Lighthouse report.
 - Discuss how the accessibility results influenced the final design.
6. **Demonstrate the application in a video recording.**
 - The video should be 3 to 8 minutes in length.
 - It should include your face, voice and student ID.
 - It should show the working application, dashboard, data-driven features, alerts, reporting views, observability metrics, Playwright tests, JMeter results, Lighthouse accessibility results, GitHub homepage and commits.
 - It should clearly explain how the system works and how the data flow supports the Wordle and Word Search builder.
7. **Produce a working technical submission.**
 - Submit the project code as a zip file.
 - Include your GitHub repository link.
 - Remove `node_modules` before uploading.
8. **Technical expectations:**
 - Use React, Next.js and server-side development practices appropriately.
 - Apply modular and reusable code where possible.

- Ensure code readability, maintainability and reliability.
- Demonstrate a data-driven architecture that can support the final live presentation in Assessment 4.

Project continuity

This assessment forms the data and reporting stage of the broader project. Assessment 1 established the frontend design and usability layer. Assessment 2 added the backend, API, database and Docker implementation. Assessment 3 now builds on that foundation by introducing dashboard views, simulated input records, operational statistics, alerts, reporting and testing. Assessment 4 will then present the final integrated system as a live demonstration with questions and answers.

Resources and readings relevant to the assessment

The following modules and labs are especially relevant to this assessment:

- Module 8: Testing lecture and lab activities covering robustness, load and accessibility testing
- Module 9: Instrumentation and observability

Other relevant resources include:

- Next.js documentation
- React documentation
- Database, ORM and API documentation
- Lecture and tutorial materials on dashboard design, reporting interfaces, data interpretation, observability and system feedback

Assessment criteria

The rubric for this assessment will focus on dashboard quality, database persistence, operational statistics, alerts, reporting views, observability, testing, data presentation, technical quality, and the video demonstration. The detailed rubric will be developed after the assessment brief is finalised.

Submission details

Submit:

- A zip file containing your project code and GitHub repository link. Remove `node_modules` before uploading.
- A video recording (3 to 8 minutes maximum) showing your student ID, face and voice, and demonstrating the dashboard, data-driven features, alerts, reporting views, observability metrics, Playwright tests, JMeter results, Lighthouse accessibility results, GitHub homepage and commits.

In keeping with La Trobe University policy, all assignments are to be submitted in Moodle via Turnitin.

To be accepted, your assessment submission must generate a similarity score (you are responsible for checking this). Submitting in Word or PDF format is the best way to do this. If your submission does not generate a similarity score, it cannot be checked for plagiarism and therefore will not be marked.

AI acknowledgement

If AI use is acceptable in this assessment, you must complete and submit an 'AI acknowledgement' with your work. Failure to submit an acknowledgement when required may constitute a breach of academic integrity. You will find the AI acknowledgement on the Assessments page.

References

- Minimum five academic or industry sources
- APA 7th ed. style

Last modified: Monday, 13 July 2026, 3:50 PM

[← Assessment 2: 3-day short extension](#)

Jump to activity



[Assessment 3 Marking criteria and rubric →](#)

You are logged in as [Isaac Riley Lambert](#) ([Log out](#))

[Emergency information](#) [Child Safety](#) [Contacts](#) [Site map](#) [Accessibility](#) [Privacy](#) [Copyright and disclaimer](#)

© Copyright 2026

La Trobe University. All rights reserved. [La Trobe University CRICOS Provider Code Number 00115M](#)